

Bloque B1

Servicio A — Autenticación

Laboratorio. Construirás el servicio que registra usuarios y emite credenciales (JWT). Duración aproximada: 3 horas.

Qué vas a construir

El **Servicio A** corre en el puerto **3001** y expone dos endpoints: `POST /registro` (crea un usuario guardando su contraseña de forma segura) y `POST /login` (verifica las credenciales y entrega un **JWT**, la credencial que el Servicio B exigirá más adelante).

Dos conceptos que NO son lo mismo

bcrypt protege la contraseña en la base de datos con un hash *irreversible*. Sirve para guardar y luego comparar, nunca para 'desencriptar'.

JWT es una credencial *firmada* que prueba quién eres ante otro servicio. Su contenido es **LEGIBLE** (Base64); la firma garantiza que nadie lo alteró, no que sea secreto. Nunca guardes datos sensibles dentro de un JWT.

Paso 0 — Crear el proyecto e instalar dependencias

Desde la carpeta `servicio-auth`, inicializa el proyecto e instala lo necesario:

```
cd servicio-auth
npm init -y
npm install express mysql2 bcrypt jsonwebtoken cors dotenv
```

- **express**: el framework del servidor.
- **mysql2**: driver de MySQL con soporte de Promesas (el paquete `mysql` viejo está sin mantenimiento; no lo uses).
- **bcrypt**: hashing de contraseñas.
- **jsonwebtoken**: crear y verificar JWT.
- **cors**: permitir que el cliente (otro origen) llame a este servicio.
- **dotenv**: leer variables del archivo `.env`.

Paso 1 — La base de datos

Importa el archivo `db/schema.sql` en tu MySQL. Crea la base `awos_u3` con la tabla `usuarios` (que usa este servicio) y `tareas` (que usará el Servicio B). La columna `password` guardará el *hash*, no la contraseña en texto.

Paso 2 — Construye el servicio fragmento por fragmento

Sigue los fragmentos en orden. Lee la explicación **antes** de escribir cada uno.

Fragmento 1 — Variables de entorno (`.env`)

Archivo: servicio-auth/.env

Copia `.env.example` a `.env` y ajusta tus credenciales de MySQL. El `JWT_SECRET` debe ser una cadena larga y aleatoria. **Apúntala bien:** el Servicio B necesitará exactamente la misma.

```
PORT=3001
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=awos_u3
JWT_SECRET=cadena-larga-aleatoria-igual-en-ambos-servicios
```

Fragmento 2 — Conexión a MySQL con un pool

Archivo: servicio-auth/src/db.js

Usamos un **pool** de conexiones, no una conexión única. Un pool reutiliza conexiones y maneja varias peticiones a la vez; una conexión única se convierte en cuello de botella y si se cae, tumba el servicio. Importamos `mysql2/promise` para poder usar `async/await` en las consultas.

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;
```

Fragmento 3 — Registro de usuario (POST /registro)

Archivo: servicio-auth/src/auth.routes.js

Validamos que lleguen los tres campos (nunca confiamos en el cliente). Luego `bcrypt.hash` convierte la contraseña en un hash irreversible. Guardamos con `pool.execute` usando **placeholders** (?): así el driver separa el SQL de los datos y previene inyección SQL. Nunca concatenes valores directamente dentro del texto del SQL.

Respondemos **201** (creado) y nunca devolvemos el hash. Si el email ya existe, MySQL lanza `ER_DUP_ENTRY` y lo traducimos a **409**.

```

router.post('/registro', async (req, res, next) => {
  try {
    const { nombre, email, password } = req.body;
    if (!nombre || !email || !password) {
      return res.status(400).json({ error: 'nombre, email y password son obligatorios' });
    }
    const hash = await bcrypt.hash(password, 10);
    const sql = 'INSERT INTO usuarios (nombre, email, password) VALUES (?, ?, ?)';
    const [resultado] = await pool.execute(sql, [nombre, email, hash]);
    res.status(201).json({ id: resultado.insertId, nombre, email });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') {
      return res.status(409).json({ error: 'El email ya esta registrado' });
    }
    next(err);
  }
});

```

Fragmento 4 — Login y emisión del JWT (POST /login)

Archivo: servicio-auth/src/auth.routes.js

Buscamos al usuario por email. Si no existe O la contraseña no coincide, devolvemos el mismo **401** genérico: no revelamos cuál de los dos falló (eso ayudaría a un atacante a adivinar qué emails existen).

`bcrypt.compare` vuelve a hashear la contraseña recibida y la compara con el hash guardado. Si todo está bien, `jwt.sign` emite el token: el *payload* (id, email) que B necesitará, firmado con `JWT_SECRET`, y con expiración de 1 hora. Un token sin expiración es mala práctica.

```

router.post('/login', async (req, res, next) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ error: 'email y password son obligatorios' });
    }
    const [filas] = await pool.execute(
      'SELECT id, nombre, email, password FROM usuarios WHERE email = ?', [email]);
    if (filas.length === 0) {
      return res.status(401).json({ error: 'Credenciales invalidas' });
    }
    const usuario = filas[0];
    const ok = await bcrypt.compare(password, usuario.password);
    if (!ok) return res.status(401).json({ error: 'Credenciales invalidas' });

    const token = jwt.sign(
      { id: usuario.id, email: usuario.email },
      process.env.JWT_SECRET,
      { expiresIn: '1h' });
    res.json({ token,
      usuario: { id: usuario.id, nombre: usuario.nombre, email: usuario.email } });
  } catch (err) { next(err); }
});

```

Fragmento 5 — Armar el servidor y arrancarlo

Archivo: servicio-auth/src/index.js

Aquí juntamos todo. El **orden** de los middleware importa: se ejecutan de arriba hacia abajo. `cors` y `express.json()` primero, luego las rutas, y al final el manejador de errores de 4 argumentos. Las

rutas hacen `next(err)` y este manejador central decide cómo se ve un error 500, sin filtrar detalles internos al cliente.

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();
const authRoutes = require('./auth.routes');

const app = express();
app.use(cors());
app.use(express.json());
app.use('/', authRoutes);

app.use((err, req, res, next) => {
  console.error('[AUTH][ERROR]', err);
  res.status(500).json({ error: 'Error interno del servidor' });
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => console.log(`Servicio A en http://localhost:${PORT}`));
```

Paso 3 — Probar

Arranca el servicio **desde dentro de su carpeta** y pruébalo con Thunder Client, REST Client o el cliente HTML:

```
cd servicio-auth
node src/index.js
# Debe imprimir: Servicio A en http://localhost:3001
```

Error muy común: 'secretOrPrivateKey must have a value'

Si el login devuelve 500 con ese mensaje, casi siempre es porque arrancaste Node desde la carpeta equivocada. `dotenv` busca el archivo `.env` en el directorio donde ejecutas `node`, no junto al archivo `.js`. Solución: entra primero a la carpeta del servicio (`cd servicio-auth`) y arranca desde ahí. Verifica también que existe el archivo `.env` (no solo `.env.example`).

Resultado esperado de este bloque

- `POST /registro` crea un usuario y responde 201 (409 si el email se repite).
 - `POST /login` con credenciales correctas devuelve un token JWT.
 - Credenciales incorrectas devuelven 401.
- Guarda un token de prueba: lo necesitarás para el Servicio B en el bloque B3.